



Lecture 4: Creating Reactive Variables

INTERVIEW QUESTION

[> CORE](#)

How do you create reactive variables in Svelte?

You change a variable in your script, but the screen doesn't update. Why, and how do you make it update?

When building an application, you often have data that changes over time, like a user's score, a toggled dark mode setting, or text typed into an input field.

The Problem with Plain Variables

Imagine you have a button that increases a counter. You write `let count = 0;` in your script, and in your click function you do `count = count + 1`.

In regular JavaScript, if you change `count = 1`, does the `<h1>` tag displaying that number on the screen automatically update? No. You would have to manually find the HTML element (e.g., `document.getElementById`) and forcefully update its text every single time the number changes. This is exhausting and error-prone.

We need a system where the HTML "listens" to our variables. When the variable changes in the script, the HTML should update itself automatically without us writing manual DOM manipulation code. This automatic synchronization between your variables and your screen is called **reactivity**. Reactivity ensures that your user interface always perfectly reflects the current state of your data.

How to Tell Svelte About Reactivity

If regular JavaScript doesn't have reactivity, how does Svelte know which variables should trigger screen updates?

Should we wrap every single variable in a heavy, complex JavaScript object like `new ReactiveVariable(0)` and call special methods like `count.set(1)` to change them? Other frameworks do this, but it makes writing simple math and logic very tedious.

We want to write normal JavaScript, like `count++`, but we need a way to tell the Svelte compiler, "Hey, this specific variable is special. Please add the secret reactivity code to it when you build my app." In Svelte 5, these special compiler symbols are called **runes**. Runes are just visual flags that tell the compiler to upgrade a normal variable into a reactive one.

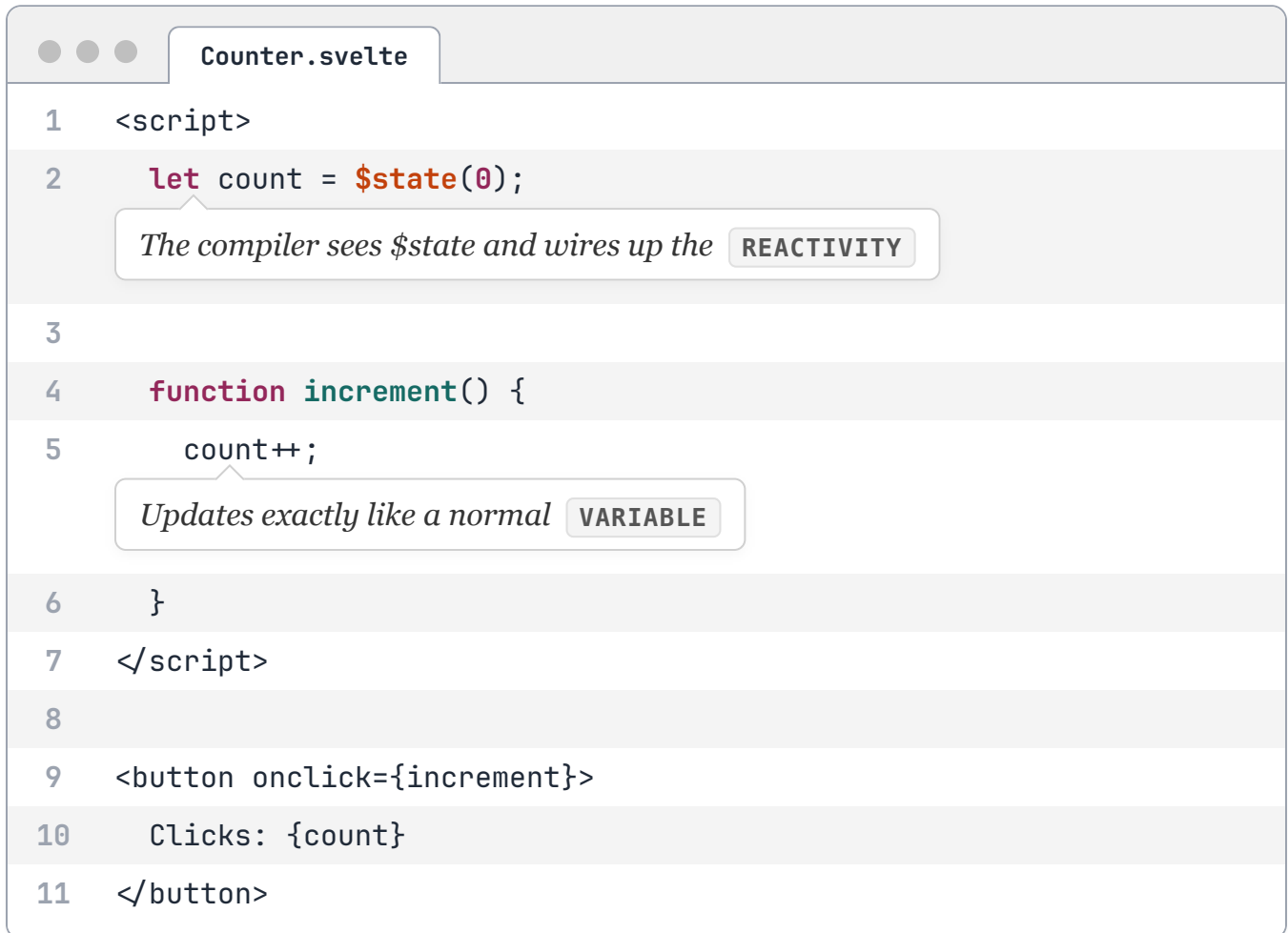
KEY TAKEAWAY

Runes are not JavaScript functions; they are instructions for the Svelte compiler.

Because runes are compiler instructions rather than runtime library functions, you don't even need to import `$state`. It acts as a built-in keyword in the Svelte language.

The `$state` Rune

To make a variable reactive, you initialize it with the `$state` rune.



```
1 <script>
2   let count = $state(0);
3
4   function increment() {
5     count++;
6   }
7 </script>
8
9 <button onclick={increment}>
10   Clicks: {count}
11 </button>
```

The compiler sees `$state` and wires up the **REACTIVITY**

Updates exactly like a normal **VARIABLE**

When the `increment` function runs and `count` increases, Svelte automatically detects the assignment (`=` , `++` , `+=`) and precisely updates only the part of the HTML where `{count}` is rendered.

Deep Reactivity

If you pass an array or an object into `$state`, Svelte makes it *deeply* reactive.



```
1 <script>
2   let todos = $state([ { text: 'Buy milk', done: false } ]);
3
4   function completeFirst() {
5     todos[0].done = true;
6   }
7 </script>
```

Triggers an update deep inside the **OBJECT**

Modifying a property deep inside the object, or calling methods like `todos.push()`, will automatically trigger the UI to update. Svelte wraps the object in a JavaScript Proxy behind the scenes, watching for any changes to its properties.

INTERVIEW TIP

To impress the interviewer: Contrast Svelte 5's `$state` with React's `useState`. Point out that Svelte's approach allows you to update variables directly (`count++`) rather than requiring a setter function (`setCount(count + 1)`), resulting in much cleaner, more natural JavaScript.

Summary

To create reactive variables in Svelte, you use the `$state` rune. This tells the compiler to automatically update the user interface whenever the variable is reassigned or modified. Because it's a compiler feature, you interact with the reactive data just like normal JavaScript variables, without needing special setter functions.

	SVELTE 5 (<code>\$state</code>) (Rune Reactivity)	REACT (<code>useState</code>) (Hook Reactivity)
Updating	Direct assignment (<code>count++</code>)	Setter function (<code>setCount(count + 1)</code>)
Deep Objects	Deeply reactive by default	Requires spreading/creating new objects
Mental Model	Compiler upgrades regular variables	Framework re-runs the entire function